Lesson 13: A Hit That Reacts

Unity

VIDEO GAME CREATION

This week: getting hit finally LOOKS like getting hit. A brief flash, then a small explosion, before the enemy disappears.

Open your own [YourName]Game project from Lesson 12 - don't start a new one.

Important: your score, lives, wave, and win/lose logic should all still work exactly as before. Today only changes how the enemy's removal LOOKS, not when your game-state actually changes.

Setup

1. Open your [YourName]Game project from Lesson 12.
2. Create a small Explosion prefab: a GameObject with a SpriteRenderer (Sprite = explosion_placeholder.png) and a script that calls Destroy(gameObject, 0.3f) in Start() - it cleans itself up automatically, no help needed from anything else.
3. Import enemy_invader_white.png (this lesson's Assets folder) into your project - it's the same enemy silhouette as Lesson 4's sprite, pre-recolored solid white, used below for the hit flash.

Level 1 - Minimal (MVP)

On every hit, the enemy briefly flashes, and a small independent explosion effect appears and cleans itself up. Score, lives, wave, and win/lose logic are all unaffected.



Everything in the top box happens the instant the hit is detected. Only the enemy's own removal - the very last step - waits for the pause to finish.

4. On the Enemy prefab's script, find the function that currently detects a bullet hit (from Lesson 4) - it likely calls GameManager.RegisterKill() and Destroy(gameObject) immediately.
5. Add public GameObject explosionPrefab, public SpriteRenderer spriteRenderer, and public Sprite flashSprite fields, and assign them in the Inspector (flashSprite = enemy_invader_white.png).
6. Rewrite the hit-handling as a Coroutine (code below) - Unity's first Coroutine this course - so it can pause partway through without freezing the whole game.
7. Inside the Coroutine: Instantiate the explosion immediately (independent of everything else), swap spriteRenderer.sprite to flashSprite, then call the existing GameManager.RegisterKill() right away too - score/wave/win-check must still happen at exactly the same instant as last week. Only AFTER that, yield return new WaitForSeconds(0.1f), then Destroy(gameObject) - that's the only line actually delayed. A color tint (spriteRenderer.color) won't work here: it multiplies the sprite's existing color, and multiplying any color by white leaves it completely unchanged - a genuinely white sprite is the only way to get a visible white flash on colored source art, the same reason Lesson 2's Recolor swaps sprites instead of tinting them.
8. Change the hit-detection code to call StartCoroutine(HandleHit()) instead of calling the old logic directly.
9. Press Play and play through a full round - each hit should flash the enemy white briefly, and a small burst should appear at that spot for a moment.

```
// Enemy's script (revised):
public GameObject explosionPrefab;
public SpriteRenderer spriteRenderer;
public Sprite flashSprite;

void OnTriggerEnter2D(Collider2D other) {
    if (other.CompareTag("Bullet")) {
        Destroy(other.gameObject);
        StartCoroutine(HandleHit());
    }
}

IEnumerator HandleHit() {
    Instantiate(explosionPrefab, transform.position, Quaternion.identity);
    spriteRenderer.sprite = flashSprite;
    GameManager.RegisterKill();
    yield return new WaitForSeconds(0.1f);
    Destroy(gameObject);
}
```

Alternate Path: Visual Scripting

The same flash-then-explode idea works in a Script Graph, using a Wait For Seconds node in place of a C# Coroutine.

10. On the enemy's hit-handling graph, add an Instantiate node for the explosion prefab right at the start (unconditional, runs immediately).
11. Set the SpriteRenderer's Sprite to flashSprite (enemy_invader_white.png) - not its Color, which would multiply against the enemy's existing color and produce no visible change - then keep your existing RegisterKill node running immediately after that, unchanged. Add a Wait For Seconds (0.1) node AFTER RegisterKill, with only the Destroy node moved to run after the wait - score/wave/win-check must not be delayed, only the enemy's actual removal.

Level 2 - Complete (Alpha/Beta)

The flash and explosion play correctly on every single hit, with no leftover explosions, no enemies stuck mid-flash, and no change to score/lives/wave timing.

- Play a full round: every single hit shows a flash and an explosion, with no enemy left stuck mid-flash.
- Confirm score, wave-clear, and win/lose all still happen at exactly the moments they did last week - yield only pauses this ONE Coroutine, nothing else in the game pauses.
- Confirm explosion prefabs never pile up in the Hierarchy - each one destroys itself via Destroy(gameObject, 0.3f), with no help from the Enemy.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Visual: instead of one flat instant swap, alternate spriteRenderer.sprite between flashSprite and the enemy's normal sprite 2-3 times across the 0.1 seconds (with tiny waits between) for a flicker instead of a single snap.
- Visual: scale the explosion prefab up slightly over its lifetime with a small script using Time.deltaTime, so it feels like it's expanding.

- Structure: notice the explosion prefab knows nothing about score, lives, or waves - it's a fully self-contained object, the same discipline as this course's bullets and enemies.

If Something's Wrong

- Enemy vanishes instantly, no flash visible: confirm the Coroutine is actually being started (StartCoroutine(HandleHit())), not the old code still calling RegisterKill()/Destroy() directly.
- Enemy disappears but never visibly turns white: confirm flashSprite is actually assigned in the Inspector to enemy_invader_white.png, not left empty or still pointing at the normal sprite.
- Explosion never appears: confirm explosionPrefab is actually assigned in the Inspector, not left empty.
- Two bullets can double-trigger the same enemy: confirm other.gameObject is destroyed at the top of OnTriggerEnter2D, before StartCoroutine runs, so a second overlapping bullet can't also collide.

Want More?

Student Resource List > Unity section, or learn.unity.com